CAPABILITIES

HelixCode TUI Capabilities Guide

Field	Value
Revision	2
Created	2026-06-14
Last modified	2026-09-23
Status	active
Status summary	—
Issues	`docs/Issues.md`
Issues summary	`docs/Issues_Summary.md`
Fixed	`docs/Fixed.md`
Fixed summary	`docs/Fixed_Summary.md`
Continuation	`docs/CONTINUATION.md`

This guide documents the newly-wired interactive capabilities of the HelixCode terminal UI (`helix-tui`). **Anti-bluff posture (CONST-035 / Article XI §11.9):** every trigger below was cross-checked against the real wiring code; each capability cites the source symbol and file path that makes it real. Capabilities that are *not* fully wired are explicitly marked **PLANNED** rather than available.

The TUI binary is built and run as:

```
cd helix_code
go build -o bin/helix-tui ./applications/terminal_ui/
./bin/helix-tui
```

Provider API keys are sourced from the environment (see §9 Providers from the environment). A common pattern is `source ~/api_keys.sh` before launching.

> **Skills documentation set:** this document's §4 and §10.5 cover the wiring of HelixCode's skill system; for an operator-facing how-to (installing a skill, directory tiers, troubleshooting) and real gotchas from this build, see `docs/guides/skills_user_manual.md` and `docs/guides/skills_faq.md`. For diagrams of the shared skills library's internal architecture, the registration path, the measured

shadowing curve, and the trust-tier/capability-enforcement flow, see `docs/architecture/skills_architecture.md`.

Table of contents

- 1. Streaming responses
- 2. MCP tools (filesystem server)
- 3. LSP diagnostics
- 4. Skills
- 5. Plugins
- 6. The agentic tool loop and read-only safety guard
- 7. The Helix Agent ensemble (visible members)
- 8. Selecting a model in the TUI
- 9. Providers from the environment via LLMsVerifier
- 10. HelixAgent full-capacity provider
- 10.5 Skills capability model — cross-consumer parity (HelixAgent)
- 11. Screenshots

---

1. Streaming responses

What it is. The TUI streams the model's final answer token-by-token instead of waiting for the whole response, so output appears as it is produced.

How it is wired. The chat submit path drives `agent.RunToolLoopStream(ctx, provider, registry, history, opts, onFinalChunk)` (`internal/agent/tool_loop.go:175`), called from the submit handler in `applications/terminal_ui/main.go:1854`. The final-chunk callback renders each streamed segment as it arrives.

How to trigger it. Just type a prompt into the chat input and press Enter — streaming is the default path.

Example prompt.

```
Summarize what the internal/llm package is responsible for.
```

What to expect. The answer renders progressively. When the selected model supports agentic tools, the loop may first run read-only tools (see §6) and then stream the final answer.

---

2. MCP tools (filesystem server)

What it is. HelixCode wires Model Context Protocol (MCP) servers into the agent tool loop. The project ships one server: the official read-only filesystem MCP server, rooted at the repository.

How it is wired. At startup the TUI loads and merges MCP config via `mcp.LoadMerged(mcpUserPath, ".helixcode/mcp.yml")` (`applications/terminal_ui/main.go:345`) and registers the manager onto the tool registry with `reg.RegisterMCPManager(mgr)` (`main.go:359`). The startup log prints the real count: `TUI: MCP wired — %d tool(s) registered from %d server(s)` (`main.go:361`).

The shipped server. `.helixcode/mcp.yml` declares a single server named `fs`, running `npx -y @modelcontextprotocol/server-filesystem <repo-root>` over stdio, with `readOnly: true` so every tool it exposes is marked `approval.LevelReadOnly` (required for the TUI's read-only-only loop to use them). The config comment lists the read-only tools the server exposes: `read_file`, `read_text_file`, `list_directory`, `directory_tree`, `search_files`, `get_file_info`, `list_allowed_directories` (and others the upstream server provides). The exact tool set and count come from the live server at registration time — HelixCode does not hardcode a number.

> **Prerequisite:** `npx` (Node.js) on `PATH`, so the filesystem MCP server can be launched.

Example prompt.

```
Use the fs MCP tool to list the files in internal/llm.
```

What to expect. The agent issues an MCP tool call (e.g. `list_directory`) and the tool trace shows the real directory listing returned by the server.

---

3. LSP diagnostics

What it is. Read-only Go diagnostics exposed to the agent as tools, backed by the Go language server.

How it is wired. `tools.WireLSP(reg, repoDir, ...)` registers two read-only tools, `lsp_get_diagnostics` and `lsp_analyze_diagnostic` (`applications/terminal_ui/main.go:335`); the startup log confirms `TUI: LSP diagnostics tools wired (lsp_get_diagnostics, lsp_analyze_diagnostic)` (`main.go:336`). Both are registered at `LevelReadOnly`, so they pass the read-only agent loop.

Prerequisite: `gopls` on `PATH`. When `gopls` is absent the LSP tools degrade gracefully (the diagnostics call returns no results rather than crashing the chat).

Example prompt.

```
Run lsp_get_diagnostics on internal/llm and tell me if there are any probl
```

What to expect. With `gopls` available, the agent calls `lsp_get_diagnostics` and reports the real diagnostics; with `gopls` absent, it reports that no diagnostics were available.

---

4. Skills

What it is. Markdown "skills" (`*.md` files with a trigger front-matter) whose trigger regexes match a user prompt; on a match the skill body *replaces* the outgoing prompt sent to the model. As of HXC-159 Phase P5 (T-P5.03 + T-P5.05), skill discovery is **unified across every HelixCode front-end** (CLI, TUI, desktop) through a single, front-end-neutral directory-resolution module — before this fix the TUI/desktop scanned only `$HOME/.helix/skills` while the CLI scanned only `os.UserConfigDir()/helixcode/skills`, so a skill installed via one front-end's documented path was invisible to the other two. See `docs/guides/skills_directory_migration.md` for the full migration narrative.

How it is wired. At startup the TUI calls `agent.LoadSkillsAndDispatcher(agent.AllSkillsDirs(".helix/skills"))` (`applications/terminal_ui/main.go:380-381`). `agent.AllSkillsDirs` (`internal/agent/skill_dirs.go:125`) returns the FULL ordered directory list every front-end now scans, **lowest precedence first**:

1. **Legacy user tier** — `$HOME/.helix/skills` (`agent.LegacyUserSkillsDir`, `internal/agent/skill_dirs.go:47`) — the pre-unification TUI/desktop directory, retained as a read-compatibility tier so nothing installed there before the fix goes missing.
2. **Canonical user tier** — `os.UserConfigDir()/helixcode/skills` (`agent.CanonicalUserSkillsDir`, `internal/agent/skill_dirs.go:27`) — the forward-looking, cross-platform-correct location every front-end should target for new installs (mirrors the CLI's sibling `.../helixcode/commands` directory and respects `XDG_CONFIG_HOME`/`%AppData%`/etc. conventions).
3. **External tier** — `.claude/skills` by default, overridable via the `HELIXCODE_EXTERNAL_SKILLS_DIR` environment variable (`agent.ExternalSkillsDir`, `internal/agent/skill_dirs.go:102`) — skills *provisioned* by something other than a user hand-authoring files

directly (e.g. `scripts/register_skills.sh`, which links constitution-owned governance skills into `.claude/skills/` by reference).
4. **Project tier** — `.helix/skills` (the caller-supplied `projectDir` argument) — highest precedence, so a project-specific skill overrides a same-named user/external/built-in skill.

Each directory is loaded via `commands.SkillLoader` in ascending-precedence order onto one shared `commands.SkillRegistry`, so a later tier's same-named skill overrides an earlier tier's (`internal/agent/skill_activation.go:23`, `LoadSkillsAndDispatcher`). On submit, `agent.DispatchSkill(tui.skillDispatcher, message)` (`applications/terminal_ui/main.go:1743`) matches the prompt against every loaded skill's trigger regex and, on a match, substitutes the rendered skill body for the outgoing LLM prompt (`internal/agent/skill_activation.go:58`).

Scope note (anti-bluff). This TUI-level skill system is a lightweight trigger-regex-to-prompt-substitution mechanism — it does **not** implement the fuller Agent Skills capability model (declared `allowed-tools`, content-hash provenance pinning, cross-consumer parity conformance) that `submodules/helix_agent`'s dedicated `internal/skills` package and the shared `submodules/skills` package provide for HelixAgent's agentic loop; see §10.5 below for that model and the cross-consumer parity note.

Shipped skill. `.helix/skills/explain-arch.md` — front-matter trigger `explain (?P<topic>\w+) architecture`, which captures a `topic` variable and renders an architecture-explanation prompt for that subsystem.

Example prompt.

```
explain ensemble architecture
```

What to expect. The `explain-arch` trigger matches, `topic` resolves to `ensemble`, and the model receives the rendered architecture-explanation template (focused on the ensemble subsystem) rather than the raw words you typed.

---

5. Plugins

What it is. Out-of-process plugins invoked from a prompt with the `@plugin:<name> <action>` syntax. HelixCode ships one read-only demo plugin, `sysinfo`.

How it is wired. At startup `plugins.LoadPlugins(context.Background(), "plugins")` (`applications/terminal_ui/main.go:388`) loads the `plugins/` directory. On submit, `plugins.MaybeRunPlugin(context.Background(),`

`tui.pluginLoader, message`) (`applications/terminal_ui/main.go:1723`) parses the `@plugin:<name> <action>` prefix and runs the matching plugin.

Shipped plugin. `plugins/sysinfo/manifest.yaml` declares `name: sysinfo`, `entrypoint: main`, `sandbox: true`, `capabilities: [info]`. The executable `plugins/sysinfo/main` supports a single action, `info` (default), and emits `uname`, the short git HEAD, and the hostname — strictly read-only (no writes, no network, no host power management per CONST-033).

Example prompt.

```
@plugin:sysinfo info
```

What to expect. The plugin output appears in chat: the `sysinfo plugin v1.0.0` banner, real `uname` output, the repo's short git HEAD, and the hostname. An unknown action exits non-zero with `sysinfo: unknown action ... (supported: info)`.

---

6. The agentic tool loop and read-only safety guard

What it is. Rather than a pure chat client, the TUI drives a multi-turn agentic loop: the model can request a tool, the loop executes it, feeds the result back, and continues until a final answer. Only **read-only** tools are reachable.

How it is wired. The TUI builds a read-only tool registry once at startup and explicitly registers `git_status` (`applications/terminal_ui/main.go:323`), alongside the auto-registered read-only `fs_read`, `glob`, and `grep`, plus the LSP and MCP read-only tools (`main.go:366` logs `TUI: agentic tool registry ready (read-only: git_status, fs_read, glob, grep + LSP/MCP tools)`). The submit path invokes `agent.RunToolLoopStream(...)` with `agent.ToolLoopOptions{ReadOnlyOnly: true}` (call at `main.go:1854`, the `ReadOnlyOnly: true` field at `main.go:1870`).

The safety guard. `ReadOnlyOnly: true` means the loop offers and executes **only** tools classified `approval.LevelReadOnly`; any write/shell tool the model attempts is refused. This is captured in the QA evidence: a model's `tool: shell` call is visibly rejected with *"not permitted in read-only mode"* (§11.4.133).

Available read-only tools: `git_status` (with subcommands such as `status`, `log`, `branch`, `diff`), `fs_read`, `glob`, `grep`, the LSP tools (`lsp_get_diagnostics`, `lsp_analyze_diagnostic`), and the MCP `fs` tools.

Example prompt.

```
What files are currently modified in this repo, and what was the last comm
```

What to expect. The agent autonomously calls `git_status` (e.g. default `status`, then `subcommand=log`), the tool trace shows the real modified-file list and real commit log, and the model summarizes the actual repository state — not a guess.

---

7. The Helix Agent ensemble (visible members)

What it is. A real multi-provider ensemble that fans each prompt to **every** configured cloud provider, scores their answers, and returns a voted winner. The TUI renders **every member's** response, its score, the model each member used, and a `[winner]` marker — so you see the whole ensemble, not just the final answer.

How it is wired. The ensemble meta-provider is registered as `EnsembleDisplayName = "Helix Agent ensemble"` (`internal/llm/ensemble_provider.go:49`), provider type/model id `helix-agent-ensemble` (`internal/llm/ensemble_provider.go:42,46`). It registers only when **two or more** member providers exist (`registerEnsembleProvider`, defined at `applications/terminal_ui/env_providers.go:248`, guard condition `len(members) < 2` at `env_providers.go:249`) — a single-member "ensemble" would be a pass-through and is refused. The per-member display is produced by `FormatEnsemblePanel` (`applications/terminal_ui/ensemble_render.go:21`), which — since the render logic was promoted into the shared, front-end-neutral `internal/ensembleui` package so the TUI and desktop GUI render an **identical** panel (§11.4.74 reuse-over-reimplement) — is now a thin re-export aliasing onto `ensembleui.FormatEnsemblePanel` rather than the panel-formatting logic itself.

Exact rendered strings (`ensemble_render.go`):

- Header: `ensemble: <successful>/<total> members` and, when present, `(strategy: <strategy>)`.
- Each member line: `[winner] <name> score=<NN.NN>` (the `[winner]` marker only on the selected provider).
- When per-member model metadata is present, the line is annotated `→ <model> (via LLMsVerifier)` — showing which verified model each member used.
- Each member's answer excerpt is indented beneath its line.

How to trigger it. Select the `Helix Agent ensemble` model in the `/model` picker (see §8), then send any prompt.

Example prompt.

```
What does this codebase do? Read the repo to answer.
```

What to expect. The ensemble panel lists each member (e.g. Groq, Mistral, OpenRouter) with its score, the model it used (`→ <model> (via LLMsVerifier)`), its answer excerpt, and a `[winner]` marker on the chosen one — plus the `ensemble: N/total members` header. Members may also run read-only tools (the same read-only guard from §6 applies).

> **Note:** the ensemble is slower than a single model because it fans the prompt to every member; the captured Video 2 run used ~75s per prompt.

---

8. Selecting a model in the TUI

What it is. Model selection is done through the `/model` command, which opens a picker modal — you do **not** type a model-name string into chat.

How it is wired. The chat command handler maps `/model` to `tui.showModelSelector()` (`applications/terminal_ui/main.go:2202-2204`; the function is also bound to the "Select Model" action button, `applications/terminal_ui/main.go:1587-1588`), defined at `applications/terminal_ui/main.go:2245` (`showModelSelector`). The picker lists `tui.llmManager.GetAvailableModels()`, sorted so the **Helix Agent ensemble** sorts first (flagship), then HelixAgent logical models, then the rest by provider and name. Each entry has a digit shortcut; selecting one calls `tui.selectModel(...)`.

How to trigger it.

1. Type `/model` and press Enter.
2. The "Select Model" modal lists every available model with a digit shortcut and `Provider: <p>, Context: <n>`.
3. Press the digit (or select) to choose. The status bar confirms the selection.

In the captured QA runs the digit shortcuts were: **digit 7** = strongest single model (Groq `llama-3.3-70b-versatile`), **digit 9** = `Helix Agent ensemble`. (Shortcut digits depend on which providers' keys are present, so they vary by environment.)

Other chat commands: `/help`, `/clear`, `/info` (shows the active model + model count).

What to expect. After selecting, the status bar shows the chosen model and provider, and subsequent prompts run against it. Selecting the ensemble warms its per-member working-model cache.

---

9. Providers from the environment via LLMsVerifier

What it is. Cloud providers are auto-registered from API keys present in the environment (e.g. via `source ~/api_keys.sh`). OpenAI-compatible hosted providers are sourced **dynamically from LLMsVerifier** — there is no hardcoded model list when the verifier is reachable (CONST-036 / CONST-046).

How it is wired. `registerEnvProviders` (`applications/terminal_ui/env_providers.go:100`) iterates the curated key-only provider set and registers each provider whose key is present (`llm.IsProviderKeyPresent`, called at `env_providers.go:115`), constructing via `llm.NewCloudProvider` (not the catch-all `llm.NewProvider`, which has no DeepSeek/Mistral arms). Then `buildOpenAICompatibleProviders(cfg)` (`env_providers.go:161`) sources the OpenAI-compatible providers dynamically — when the verifier is reachable the log states `TUI: OpenAI-compatible providers sourced DYNAMICALLY from LLMsVerifier` (`env_providers.go:163`); when unreachable it falls back to a hardcoded catalogue as a **degraded offline fallback only** (`env_providers.go:165`).

> **Cloud gate note (W2c-1, 2026-09-05):** cloud-provider construction is additionally gated by `llm.SetCloudEnabled(cfg.LLM.Cloud.Enabled)` (`applications/terminal_ui/main.go:287`), default **false**. When a credentialed provider's construction is refused because `llm.cloud.enabled` is `false` (the default), the TUI logs a single summary line — `TUI: N hosted provider(s) with credentials present were NOT registered because llm.cloud.enabled is false` (`env_providers.go:152`) — rather than a per-provider warning, and local routes (helixllm coder, llama.cpp, Ollama) are unaffected. Set `llm.cloud.enabled: true` in config to permit hosted providers.

Key-only cloud providers auto-registered (`envProviderCandidates`, `env_providers.go:77-86`): DeepSeek, Mistral, Groq, OpenRouter, OpenAI, Anthropic, xAI, Qwen. The recognized environment variable names (with aliases) come from `ProviderEnvAliases()` (`internal/llm/keyrecognition.go:32`), including:

Provider	Env var(s)
OpenAI	`OPENAI_API_KEY`, `ApiKey_OpenAI`
Anthropic	`ANTHROPIC_API_KEY`, `CLAUDE_API_KEY`
Gemini	`GEMINI_API_KEY`, `GOOGLE_API_KEY`, `ApiKey_Gemini`

Provider	Env var(s)
DeepSeek	`DEEPSEEK_API_KEY`, `ApiKey_DeepSeek`
Mistral	`MISTRAL_API_KEY`, `ApiKey_Mistral_AiStudio`
Groq	`GROQ_API_KEY`, `ApiKey_Groq`
xAI	`XAI_API_KEY`, `GROK_API_KEY`, `ApiKey_xAI`
OpenRouter	`OPENROUTER_API_KEY`, `ApiKey_OpenRouter`
Qwen	`QWEN_API_KEY`, `DASHSCOPE_API_KEY`, `ApiKey_Qwen`
Cerebras	`CEREBRAS_API_KEY`, `ApiKey_Cerebras`
Copilot	`GITHUB_COPILOT_TOKEN`, `COPILOT_API_KEY`

> **Scope note (anti-bluff):** Bedrock, Vertex AI, and Azure are intentionally **not** auto-registered from a bare env var — they need region/project/deployment wiring; auto-registering them from one key would produce a half-configured provider (`env_providers.go:72-76`). They remain reachable via the explicit `config.yaml` / server path.

How to trigger it. Export the keys you have (or `source ~/api_keys.sh`) before launching the TUI; present-key providers appear in the `/model` picker automatically.

What to expect. Startup logs show how many providers and models registered; unreachable providers are skipped cleanly (no fake picker entries). In the captured QA run the live provider count went 5 → 18 with the verifier populated.

---

10. HelixAgent full-capacity provider

What it is. The full-capacity HelixAgent engine (its own multi-provider engine + ensemble), consumed over its running REST server, registered as a standalone provider so its logical models appear in the picker.

How it is wired. `registerHelixAgentProvider` (defined at `applications/terminal_ui/env_providers.go:203`, doc comment from `env_providers.go:189`) resolves the base URL from `HELIXAGENT_BASE_URL` (falling back to `helixagent.DefaultBaseURL`, `env_providers.go:208-211`), constructs `helixagent.New(baseURL)`, and runs a 2-second `IsAvailable` health probe **before** registering — so the picker never lists a dead HelixAgent entry when the server isn't running (anti-bluff: a listed model is a claim it can be used, `env_providers.go:219-224`). Its logical models (`helixagent-llm`,

`helixagent-ensemble`) are sourced from HelixAgent's live `/v1/models` (no hardcoded list, per the CONST-036/037 note at `env_providers.go:199-201`).

How to trigger it.

1. Start the HelixAgent server (optionally set `HELIXAGENT_BASE_URL` to point at it; otherwise the documented default base URL is used).
2. Launch `helix-tui`. If the server is reachable, the startup log shows `TUI: registered HelixAgent provider (full-capacity engine + ensemble) at <url>`.
3. Open `/model` and select a `helixagent-llm` or `helixagent-ensemble` model.

What to expect. With the server reachable, HelixAgent's logical models appear in the picker and run against its full engine; with the server down, you instead see the honest info line *"HelixAgent not reachable … skipping"* and no dead entry.

10.5 Skills capability model — cross-consumer parity (HelixAgent)

What it is. HelixCode's TUI skill system (§4) is deliberately lightweight (trigger-regex → prompt substitution). `submodules/helix_agent` ships a **fuller Agent Skills capability model** on top of the same on-disk skill format, adding declared tool-capability enforcement, content-hash provenance pinning, and an external-source registration path — and both consumers' capability surfaces are mechanically diffed by a parity gate so the two never silently drift apart.

Declared capability enforcement (`AllowedTools`). Each skill's front-matter may declare `allowed-tools` (`submodules/helix_agent/internal/skills/types.go:16`, struct field `AllowedTools`), parsed by `ParseAllowedTools` (`submodules/helix_agent/internal/skills/types.go:179`). At the tool-call boundary, `SkillAuthorizer.Authorize` (`submodules/helix_agent/internal/skills/authorizer.go:56`) checks the calling skill's declared `AllowedTools` against the requested tool name via `skillDeclaresTool` (`authorizer.go:101`): a skill whose `AllowedTools` is non-empty and does not name the tool is **refused**; a skill with an **empty** `AllowedTools` is a deliberate backward-compatible allow (`authorizer.go:35`) — every authorization decision is recorded to an audit log (`AuditLog`, `authorizer.go:88`).

External tool-source registration. `ExternalSkillSource.RegisterWith` (`submodules/helix_agent/internal/skills/external_source.go:98`) calls `ToolRegistry.RegisterExternalToolSource` (`submodules/helix_agent/internal/services/tool_registry.go:159`) to expose an external skill

directory's skills as callable tools — the mechanism a skill loaded from outside the built-in tier uses to become invocable.

Content-hash provenance pinning. The capability manifest each consumer generates carries a `content_hash_provenance` flag (`submodules/helix_agent/internal/skillconformance/manifest.go:49`) recording whether a skill's provenance is pinned by content hash rather than by a mutable path reference — part of the same `CapabilityManifest` schema (`manifest.go:69`, `Validate` at `manifest.go:81`) both consumers populate.

Cross-consumer parity gate. Both `helix_code` and `helix_agent` ship their own `internal/skillconformance` package generating a comparable `CapabilityManifest` for their own loaded-skill surface (helix_code's report generator: `helix_code/internal/skillconformance/report.go:74`, `GenerateReport`). Gate `CM-CROSS-CONSUMER-PARITY` (HXC-159 T-P7.02, `helix_code/cmd/skillparity/main.go`) reads one manifest JSON from each consumer and diffs them via `skillparity.Diff` (`helix_code/internal/skillparity/parity.go:78`), exiting non-zero on any **unexplained** asymmetry (`skillparity.Result.OK`, `parity.go:40`) — a capability present on one consumer and silently absent from the other (rather than deliberately, mutually marked as such via `isMutuallyMarked`, `parity.go:49`) fails the gate. This is the mechanism referenced elsewhere in this document as "the cross-consumer parity gate": HXC-159 Phase P7 used it to find and close a capability gap between the two consumers before this revision.

---

11. Screenshots

The frames below are **real captured TUI frames** committed under `docs/qa/`. They are referenced here by their committed paths (relative to the meta-repo root); see the run's `README.md` in each directory for full reproduction details and verdicts.

All-providers + LLMsVerifier integration — `docs/qa/all-providers-keys-integration-20260614/`

Frame	Shows
`frames/q1-positive-cerebras-gptoss120b.png`	A positive codebase answer through a newly-wired OpenAI-compatible provider (Cerebras gpt-oss-120b).
`frames/helixagent-q2-agents-md-reads-real-files.png`	The agent reading real files (`AGENTS.md`) via read-only tools.

Frame	Shows
`frames/helixagent-q3-gitstatus-status-log-diff.png`	A real `git_status` trace (status / log / diff).

TUI ensemble videos (re-record) — `docs/qa/tui-ensemble-videos-20260614/`

Frame	Shows
`frames/video1-p3-git-status-status+log.png`	A single strongest model autonomously calling `git_status` twice (default `status` + `subcommand=log`); real modified-files list + real commit log.
`frames/video2-p1-codebase-3of4-members.png`	The ensemble panel rendering **3/4 members** (Groq/Mistral/OpenRouter), each with a real codebase answer + score; `git_status` executed.
`frames/video2-p2-agentsmd-4of4-members-shell-refused.png`	**4/4 members** using `grep`/`glob`/`git_status` to find the real `AGENTS.md`; the read-only safety guard **visibly refuses `tool: shell`** ("not permitted in read-only mode").
`frames/video2-p3-gitstatus-3of4-members.png`	**3/4 members** each giving a real git-status summary; `git_status subcommand=branch` output.

DeepSeek + strong-models run — `docs/qa/tui-deepseek-v4-strong-models-20260614/`

Frame	Shows
`frames/video1-deepseek-v4-pro-smoke-3-gitcalls.png`	A single strong model making three real `git_status` calls.
`frames/video1-deepseek-v4-pro-p3-gitstatus-diff-show.png`	`git_status` status/diff/show trace.
`frames/video2-ensemble-p1-codebase-3of4.png`	Ensemble panel, 3/4 members answering the codebase question.
`frames/video2-ensemble-p2-agentsmd-2of4-resolves.png`	Ensemble resolving `AGENTS.md`, 2/4 members.
	Ensemble git-status, 3/4 members.

Frame	Shows
frames/video2-ensemble-p3-gitstatus-3of4.png	

Sources verified 2026-06-14 (initial); re-verified 2026-09-23 (Revision 2, HXC-159 T-P8.01)

All triggers were originally cross-checked against the live wiring code as of the 2026-06-14 commit. **Revision 2 (2026-09-23, HXC-159 T-P8.01.1)** mechanically re-derived every `file:line` citation in this document against the current tree — every citation was opened at its cited line and its claimed symbol/log-string confirmed present; every drifted citation was corrected in place, and §4 (Skills) was substantively rewritten to describe the unified `agent.AllSkillsDirs` four-tier directory model HXC-159 Phase P5 (T-P5.03/T-P5.05) introduced, replacing the pre-unification two-directory description. The re-check covered: `applications/terminal_ui/main.go`, `applications/terminal_ui/env_providers.go`, `applications/terminal_ui/ensemble_render.go`, `internal/agent/tool_loop.go`, `internal/agent/skill_dirs.go`, `internal/agent/skill_activation.go`, `internal/llm/ensemble_provider.go`, `internal/llm/keyrecognition.go`, `internal/ensembleui/*.go`, `.helixcode/mcp.yml`, `.helix/skills/explain-arch.md`, `plugins/sysinfo/manifest.yaml`, plus `submodules/helix_agent/internal/skills/*.go`, `submodules/helix_agent/internal/services/tool_registry.go`, `submodules/helix_agent/internal/skillconformance/manifest.go`, `helix_code/internal/skillconformance/report.go`, and `helix_code/internal/skillparity/parity.go` for the new §10.5. Screenshot paths verified present under `docs/qa/` at the meta-repo root. Runtime signature: `qa-results/hxc159/docs_update/<run-id>/` (see the T-P8.01 completion evidence for the mechanical citation-checker output).

> **Export note (CONST-066 / CONST-063):** the synchronized `.html` and `.pdf` siblings of this document are a follow-up; this revision lands the `.md` only.